PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
Projetos de Sistemas Operacionais
1s/2026 – Prof. Lucas Reis

Laboratório 5 - Deadlocks

Este laboratório explora o conceito de deadlock (impasse) em sistemas operacionais, um problema clássico de sincronização que ocorre quando processos ou threads competem por recursos de forma circular. Você analisará um código que apresenta condições de deadlock e aprenderá a identificar e corrigir esse problema.

1. Para facilitar o desenvolvimento, os arquivos necessários para realização desse laboratório estão no Canvas com o nome de `lab5.zip`. Entretanto, eu recomendo fortemente **digitar os arquivos manualmente**. Isso te força a ler o arquivo manualmente linha a linha, auxiliando o entendimento dos programas trabalhados neste laboratório.

2. Esse documento assume que você possui a máquina virtual construída durante o processo do primeiro laboratório (`Lab0`), **incluindo a configuração do diretório compartilhado entre o host e a máquina virtual**. Caso você não tenha feito/refeito esses passos, não será possível realizar este laboratório.

Instruções de Entrega

Um conjunto de questões está presente neste questionário. Elas têm o formato a seguir:

> Exemplo
>
> **Resposta exemplo**

Responda às questões dissertativas diretamente neste PDF utilizando seu editor de preferência.

Arquivos necessários para entrega:

- O PDF modificado com suas respostas.
- Arquivos em C solicitados ao decorrer das questões neste documento (identificados **em azul**).

Atenção: todos os arquivos acima devem ser compactados em um único arquivo no formato `.zip`. A entrega deve ser feita via Canvas até às 23:59 do dia anterior à nossa próxima aula de laboratório.

1 Configuração do Ambiente

Nota importante: Devido à VM ter múltiplas CPUs, o deadlock pode não ocorrer de forma consistente. Para garantir que o deadlock aconteça, você tem duas opções:

- **Opcional 1:** Usar o comando `taskset -c 0` antes de executar o programa.
- **Opcional 2:** Configurar a VM para usar apenas 1 CPU (nas configurações da VirtualBox, reduza o número de processadores para 1).

2 Análise de um Deadlock

Nesta seção, você receberá um programa que contém dois semáforos e duas threads. O programa demonstra um problema clássico de sincronização.

ATENÇÃO: Não altere nada no código ainda! Compile e execute primeiro para observar o comportamento.

```
gcc deadlock_exemplo.c -lpthread -o deadlock_exemplo
taskset -c 0 ./deadlock_exemplo
```

Observe que o programa começa a execução, mas não termina. As threads param de produzir saída no console e o programa fica travado indefinidamente. Use `Ctrl+C` para interromper a execução.

Nota: Use `taskset -c 0` conforme explicado na seção "Configuração do Ambiente"para garantir o deadlock.

Analise o código fonte `deadlock_exemplo.c` linha a linha.

Dica: Preste atenção na ordem em que os semáforos são adquiridos em cada thread.

Explique detalhadamente por que este programa causa um deadlock.

Esse programa causa um deadlock ao atender as quatro condições de um deadlock. Nele a thread A e B querem utilizar os mesmo recursos, condição de corrida, e para isso utilizam semáforos. Porém essa sincronização foi mal implementada, causando um deadlock, onde A espera B que espera A.

O deadlock ocorre porque quatro condições necessárias são satisfeitas simultaneamente.

Identifique quais condições estão presentes neste código e explique como cada uma se manifesta na prática.

1° Exclusão mutua, é atendida de forma simbólica, representado pelas seções críticas com as instruções de escrita e aplicado a sincronização com semáforos 1 e 2, nas threads A e B.
2° Posse e espera, é atendida quando no código as threads "pegam" um dos semáforos para elas e pedem pelo outro.
3° Sem preempção, não é implementado nenhum mecanismo que interrompa a execução das threads retirando os semáforos delas, portanto não existe preempção.
4° Espera Circular, TA -> SEM1 -> TB -> SEM2 -> TA. A partir desse esquema é possível identificar a espera circular, onde a thread espera por recurso (SEM) que esta sendo "segurado" por outra thread, que por sua vez espera o recurso segurado pela inical/primeira, configura uma Espera Circular.

2.1 Correção do Deadlock

Agora que você identificou o problema, sua tarefa é corrigi-lo.

RESTRIÇÃO IMPORTANTE: Você deve modificar **apenas a estrutura de aquisição dos semáforos**. Não altere a lógica das threads, não adicione novas variáveis, não modifique os tempos de sleep e não remova nenhuma linha de código existente. A solução deve ser feita apenas reorganizando a ordem ou a estrutura de como os semáforos são adquiridos e liberados.

1. Abra o arquivo `deadlock_exemplo.c` em seu editor.
2. Identifique onde os semáforos são adquiridos em cada thread.
3. Reestruture a aquisição dos semáforos para evitar o deadlock, mantendo todas as outras operações do código intactas.
4. Compile e teste sua solução. O programa deve executar até o final sem travar.

[Entrega]: Entregue o **código-fonte** corrigido como `deadlock_corrigido.c`.

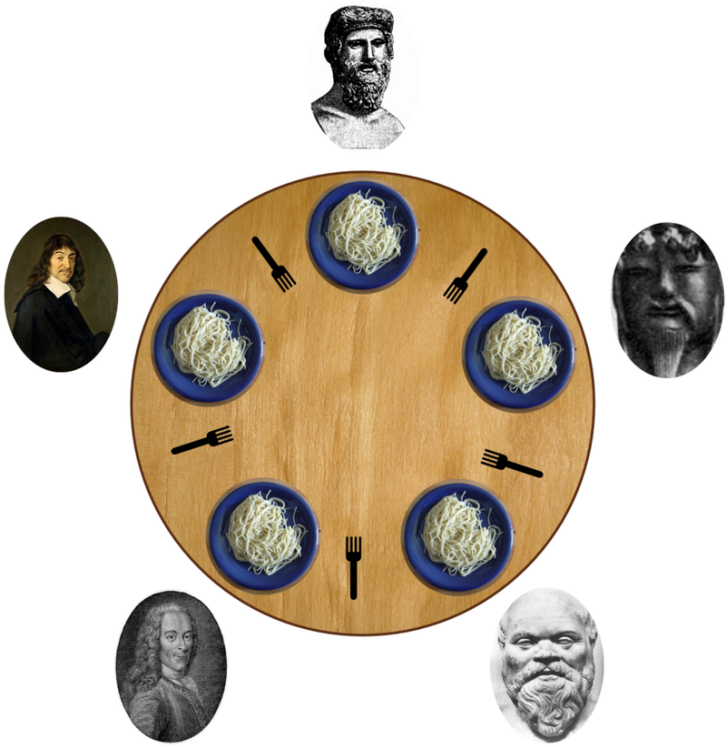
Por que sua alteração resolve o problema? Qual princípio de prevenção de deadlock você aplicou?

Resolvi o problema evitando que a espera circular ocorra ao colocar as requisições de recursos (sem_wait) em uma ordem padrão para as duas threads.
Dessa forma, evitei a espera circular ao organizar as requisições de recursos na mesma ordem para as duas threads, assim é impossivel que uma thread segure um recurso que outra pode precisar sem antes esbarrar em um recurso já ocupado.

3 O Problema do Jantar dos Filósofos

O **Problema do Jantar dos Filósofos** (Dining Philosophers Problem) é um dos problemas clássicos mais importantes da ciência da computação para ilustrar o conceito de deadlock. Embora seja apresentado de forma didática, ele representa situações reais que aparecem em sistemas operacionais e programas concorrentes.

Imagine cinco filósofos sentados em volta de uma mesa redonda. Cada filósofo tem um prato de spaghetti à sua frente e há um garfo posto entre cada par de vizinhos. A figura abaixo ilustra essa configuração:



Cada filósofo alterna entre dois estados: **pensar** e **comer**. Quando está pensando, não precisa de nenhum recurso. Quando quer comer, precisa adquirir **dois garfos** (o

que está à sua esquerda e o que está à sua direita). Após terminar de comer, o filósofo devolve os dois garfos para a mesa e volta a pensar.

Este problema é uma **analogia direta** para sistemas operacionais:

- **Filósofos** → **Processos/Threads**
- **Garfos** → **Recursos do sistema** (lock, mutex, semáforo, arquivo, etc.)
- **Comer** → **Usar os recursos**

3.1 Por que ocorre deadlock?

Vamos analisar como o deadlock pode ocorrer neste cenário. Considere que todos os filósofos decidam comer **simultaneamente** e sigam o seguinte algoritmo:

1. Pegar o garfo da esquerda
2. Pegar o garfo da direita
3. Comer
4. Devolver os dois garfos

Cada filósofo consegue pegar seu garfo da esquerda (há 4 garfos e 4 filósofos). Porém, quando tentam pegar o garfo da direita, ele já está nas mãos do filósofo vizinho. Todos os filósofos ficam esperando um garfo que nunca será liberado, pois cada um está esperando que seu vizinho devolva o garfo. **Todos os 4 filósofos ficam bloqueados infinitamente** - ocorreu um deadlock!

3.2 As Quatro Condições de Coffman

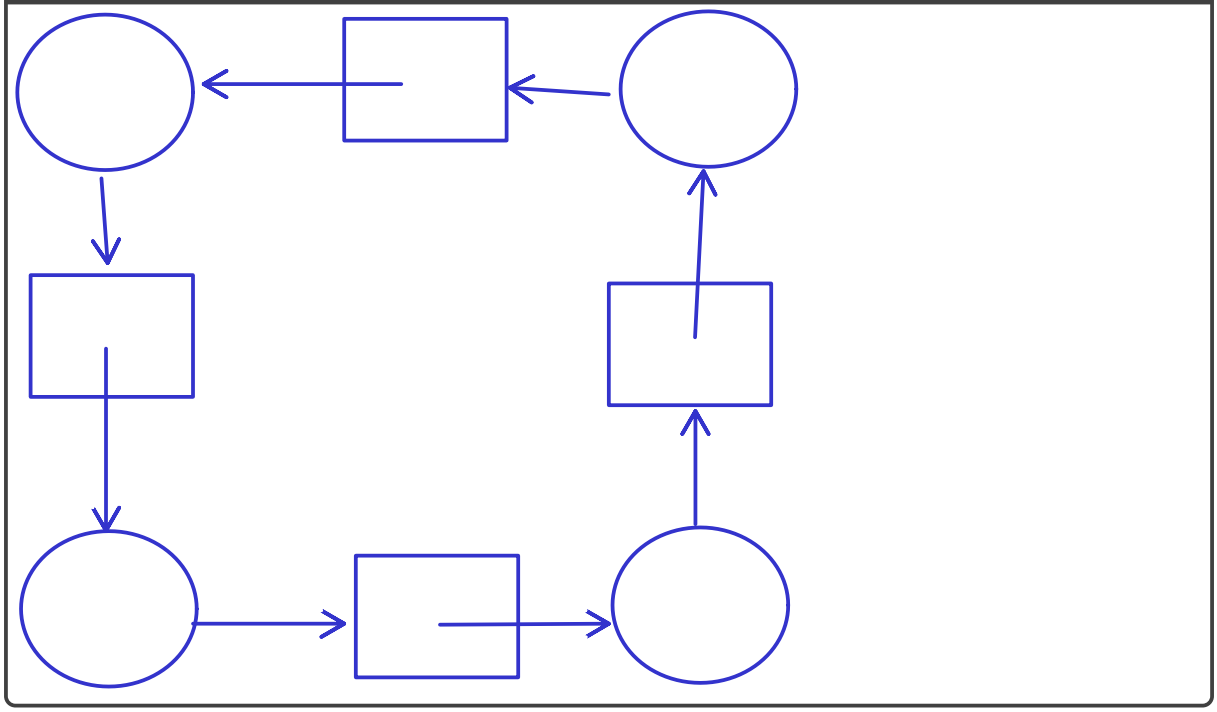
Lembre-se: um deadlock só pode ocorrer quando **todas** as quatro condições abaixo são satisfeitas simultaneamente. Vamos identificar cada uma no contexto dos filósofos:

1. **Exclusão mútua:** Os garfos são recursos que só podem ser utilizados por um filósofo por vez. Um garfo não pode ser compartilhado - se está na mão de um filósofo, outro não pode pegá-lo.
2. **Retenção e Espera:** Um filósofo segura um garfo (o da esquerda) enquanto espera pelo outro (o da direita). Ele não solta o que já tem para pegar o próximo - ele segura ambos.
3. **Inexistência de preempção:** Os garfos não podem ser tirados à força de um filósofo. Se um filósofo pegou um garfo, você não pode arrancá-lo da mão dele. O recurso só é liberado voluntariamente quando o filósofo termina de comer.
4. **Espera circular:** Os filósofos formam um ciclo de espera: o filósofo 0 espera o garfo do filósofo 1, que espera o garfo do filósofo 2, e assim por diante, até o filósofo 4 esperar o garfo do filósofo 0.

3.3 Análise com 4 Filósofos

Para consolidar o entendimento, vamos analisar um cenário com **apenas 4 filósofos**. Considere que todos os 4 filósofos decidam comer ao mesmo tempo e sigam o algoritmo descrito anteriormente (pegar esquerda, depois direita).

Desenhe o grafo de alocação de recursos para este cenário com 4 filósofos. O grafo deve mostrar os filósofos (círculos), os garfos (quadrados), e as setas indicando: (1) qual filósofo possui qual garfo, e (2) qual filósofo está esperando qual garfo. A partir do grafo, identifique onde está a espera circular que causa o deadlock.



3.4 Implementação Prática

Vamos verificar na prática se o deadlock realmente ocorre. O arquivo `filosofos.c` implementa o problema dos filósofos com 4 filósofos, onde cada filósofo tenta pegar o garfo da esquerda primeiro e depois o da direita.

ATENÇÃO: Não altere nada no código ainda! Compile e execute primeiro para observar o comportamento.

```
gcc filosofos.c -lpthread -o filosofos
taskset -c 0 ./filosofos
```

Nota: O comando `taskset -c 0` limita o programa a usar apenas uma CPU. Isso garante que o deadlock ocorra de forma consistente, pois impede que as threads executem em paralelo.

Observe que o programa começa a execução, mas não termina. Os filósofos param de produzir saída no console e o programa fica travado indefinidamente. Use `Ctrl+C` para interromper a execução.

Compare o comportamento deste programa com o primeiro exemplo de deadlock deste laboratório. Por que ambos os programas causam deadlock? O que eles têm em comum?

Ambos os programas resolvem a condição de corriga implementando a seção crítica (sincronização) de forma errada, e ao fazer isso eles atendem as 4 condições para que ocorra um deadlock.

3.5 Estratégias de Solução

Existem várias formas de resolver o problema dos filósofos. Nesta seção, apresentamos duas estratégias clássicas:

1. Hierarquia de Recursos: No exemplo original, os processos devem adquirir os garfos na mesma ordem - todos pegam o garfo da esquerda primeira. Nessa solução, os recursos são organizados em uma ordem fixada de aquisição. Todos os processos devem sempre adquirir os recursos nessa ordem. No caso dos filósofos, isso significa que todos devem pegar primeiro o garfo com **número menor** (ou maior), e depois o outro. Isso elimina a espera circular pois quebra o ciclo de espera.

2. Aquisição Atômica de Múltiplos Recursos: Um processo que precisa de vários recursos deve adquirir todos de uma vez, de forma atômica. No problema dos filósofos, isso seria equivalente a um filósofo tentar pegar os dois garfos **simultaneamente**. Se não conseguir adquirir os dois, não adquire nenhum e espera até que ambos estejam disponíveis.

Ambas as estratégias atacam uma das quatro condições de Coffman, impedindo assim que o deadlock ocorra.

3.6 Implementação da Solução

Agora é a sua vez! Utilize uma das estratégias apresentadas acima para corrigir o problema dos filósofos em `filosofos.c`.

1. Abra o arquivo `filosofos.c` em seu editor.
2. Implemente uma das estratégias de solução (hierarquia de recursos ou aquisição atômica).
3. Compile o programa: `gcc filosofos.c -lpthread -o filosofos`
4. Execute com `taskset -c 0 ./filosofos` e verifique se o programa termina sem deadlock.

[Entrega]: Entregue o **código-fonte** corrigido como `filosofos_corrigido.c`.

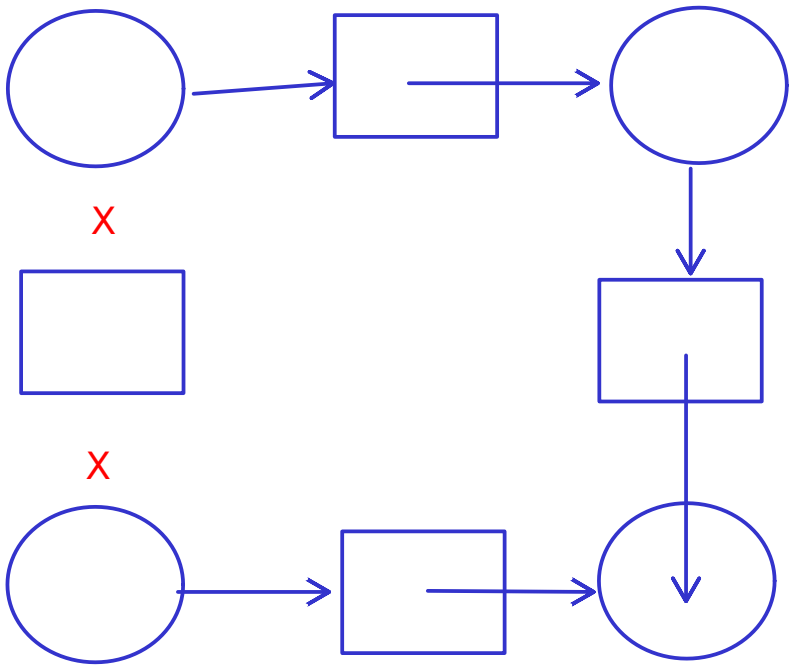
Explique qual estratégia você utilizou e como ela resolve o problema do deadlock.

Eu utilizei a hierarquia de recursos para solucionar o problema, fiz isso forçando que filósofos de id par pegassem primeiro o garfo da esquerda e depois o da direira, filósofos de id ímpar fazem o contrario, assim é impossivel que a condição da espera circular aconteça.
Essa solução também prevê a escalabilidade do algoritmo.

3.7 Análise da Solução

sr

Desenhe o grafo de alocação de recursos para a sua solução, mostrando que não há mais deadlock.



4 Observação com strace (Opcional)

Se desejar aprofundar sua compreensão, você pode usar a ferramenta `strace` para observar as chamadas de sistema relacionadas aos semáforos.

Em um terminal, execute:

```
strace -e futex ./deadlock_exemplo
```

Observe que as threads ficam bloqueadas nas chamadas `futex`, aguardando indefinidamente. Depois de corrigir o código, execute o mesmo comando e observe a diferença: as threads agora conseguem completar suas operações e o programa termina normalmente.